UNIVERSITY ASSIGNMENT · TUTORIAL REPORT · MARCH 2026

How Deep Should a Neural Network Be?

Understanding Underfitting and Overfitting in Multilayer Perceptrons

STUDENT

Banoth Venkanna

MODULE

Machine Learning / Neural Networks

DATE

March 2026

TYPE

Tutorial Assignment

CONTENTS

Table of Contents

1. Introduction
2. Background Concepts
3. Dataset and Method
4. Experiment Design
5. Results
6. Discussion
7. Ethical Considerations
8. Conclusion
9. References
10. GitHub Repository

SECTION 1

Introduction

How does the depth of a neural network affect its performance — and where exactly does "more layers" stop helping?

Machine learning models are widely used to identify patterns and make predictions from data. Among these models, neural networks are particularly powerful due to their ability to learn complex, non-linear relationships that simpler methods struggle to capture. However, designing a neural network is not as simple as it might seem, especially when it comes to deciding how many layers it should have.

A network with too few layers may fail to capture meaningful structure in the data — a problem known as *underfitting*. Conversely, a network with too many layers may memorise the training data too closely, performing poorly on unseen examples — a problem known as *overfitting*. Getting the balance right is one of the central challenges of practical machine learning.

This tutorial focuses on a single, concrete question: **How does the depth of a neural network affect its performance?** To answer this, the tutorial explores three Multilayer Perceptron (MLP) architectures of increasing depth, comparing their training and generalisation behaviour on a regression task.

> **TUTORIAL OBJECTIVE**
>
> By the end of this tutorial, the reader should be able to distinguish underfitting, good generalisation, and overfitting in practice, and understand why network depth is a critical design decision rather than a simple "more is better" knob.

SECTION 2

Background Concepts

Before examining the experiments, it is helpful to establish a clear picture of the key ideas involved. This section provides concise explanations of the core concepts referenced throughout the tutorial.

What is a Multilayer Perceptron?

A *Multilayer Perceptron* (MLP) is one of the most fundamental types of artificial neural network. It consists of an ordered sequence of layers, each made up of artificial neurons. Information flows in a single direction — from input to output, passing through one or more intermediate layers. This architecture is sometimes called a *feedforward* network. Each neuron takes a weighted sum of its inputs, adds a bias term, and applies a non-linear activation function to

produce its output.

Input Layer

The first layer. It receives the raw feature values for each sample and passes them forward. This layer does not perform any computation; it simply holds the data.

Hidden Layers

The intermediate layers between input and output. These are where the actual learning happens. Each hidden layer transforms its input into a new, more abstract representation. The number and size of hidden layers is the key architectural choice explored in this tutorial.

Output Layer

The final layer. For a regression task, this layer typically consists of a single neuron with a linear activation, producing a continuous numerical prediction.

Underfitting

Underfitting occurs when a model is too simple to capture the underlying structure of the data. An underfitting model performs poorly on both training and test data — it fails to learn the available patterns and makes imprecise, naive predictions. In MLPs, this is typically caused by too few layers or too few neurons.

Overfitting

Overfitting occurs at the opposite extreme. A model with excessive capacity tends to memorise training samples — including their noise — rather than learning generalisable patterns. Such a model performs well on training data but poorly on a held-out test set. Regularisation techniques such as dropout or weight decay are commonly used to mitigate it.

Balanced Model / Good Generalisation

A well-fitted model is complex enough to capture the true structure in the data but not so complex that it fits the noise. It achieves low error on both training and test data, with a small gap between the two. Finding this balance is the central theme of model selection.

SECTION 3

Dataset and Method

Dataset: Diabetes (scikit-learn)

The dataset used is the *Diabetes dataset* provided by the scikit-learn library. It is a regression dataset containing 442 samples, each described by 10 baseline features — such as age, sex, body mass index, average blood pressure, and six blood serum measurements — taken from diabetes patients. The target variable is a quantitative measure of disease progression one year after the baseline measurements were recorded.

Train / Test Split

The data was partitioned using an 80/20 split — 80% of samples for training and 20% held out for evaluation. This split is performed before any preprocessing to avoid information leakage from the test set into the training procedure.

Feature Scaling

All features were standardised using **StandardScaler**: the mean of each feature was subtracted and the result divided by its standard deviation, giving zero mean and unit variance. This step is important because neural networks trained with gradient descent are sensitive to the scale of input features. Without standardisation, features with larger magnitudes can dominate the gradient signal, leading to slower and less stable training.

> **NOTE ON REPRODUCIBILITY**
>
> A fixed random seed was used throughout the experiments to ensure that the train/test split and model weight initialisation were consistent across runs, making results reproducible and fair across comparisons.

SECTION 4

Experiment Design

Three distinct MLP architectures were designed, each representing a different point on the spectrum from underfitting to overfitting. The only variable between models is the network architecture — all other training conditions are identical.

Table 1 — Overview of the three experimental architectures

Model	Architecture	Approx. Parameters	Expected Behaviour	Label
Model A	(8)	~100	Shallow; limited capacity	Underfitting
Model B	(32, 16)	~900	Moderate; good generalisation	Balanced
Model C	(128, 64, 32, 16)	~13,000	Deep; risk of overfitting	Overfitting

Model A was designed as the underfitting baseline. With only eight neurons in a single hidden layer, it has very limited representational capacity. Model B represents a moderate architecture expected to generalise well. Model C was constructed to investigate overfitting behaviour; with

roughly 13,000 parameters trained on 354 samples, the model has enough capacity to memorise rather than learn.

SECTION 5

Results

The following figures illustrate the experimental outcomes. Each chart addresses a different aspect of model performance, together providing a comprehensive view of how depth affects both training behaviour and generalisation ability.

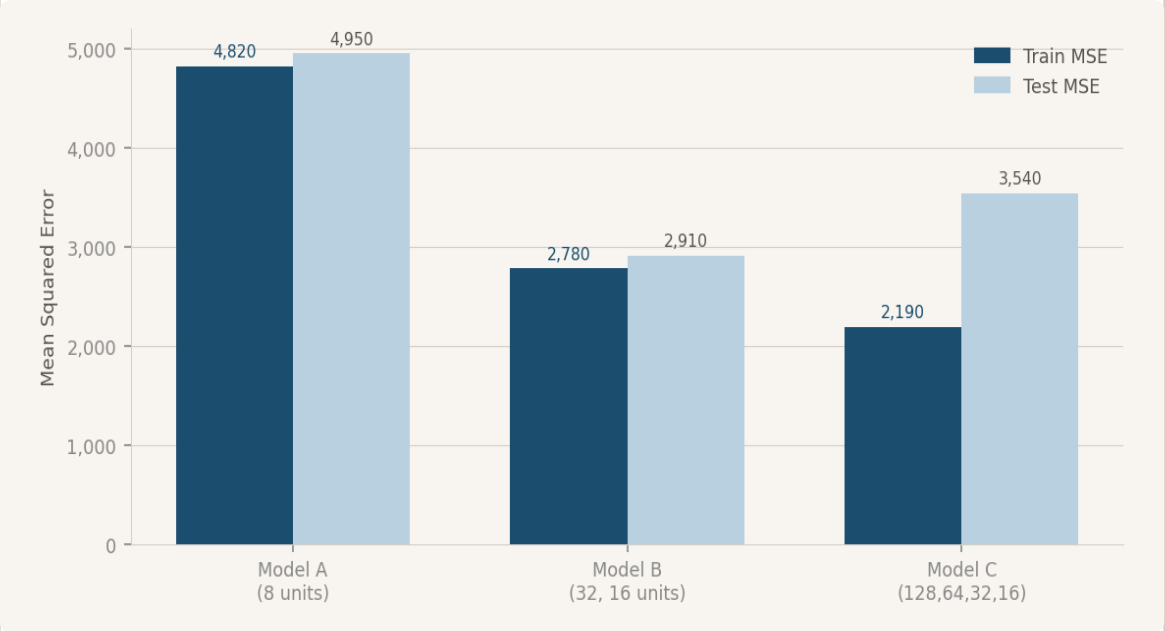

Figure 1.
Comparison of train and test MSE across the three architectures. Model A shows high error on both sets (underfitting). Model B achieves the lowest test MSE with a small train–test gap. Model C achieves lower training error but shows a noticeably wider gap, indicating overfitting.

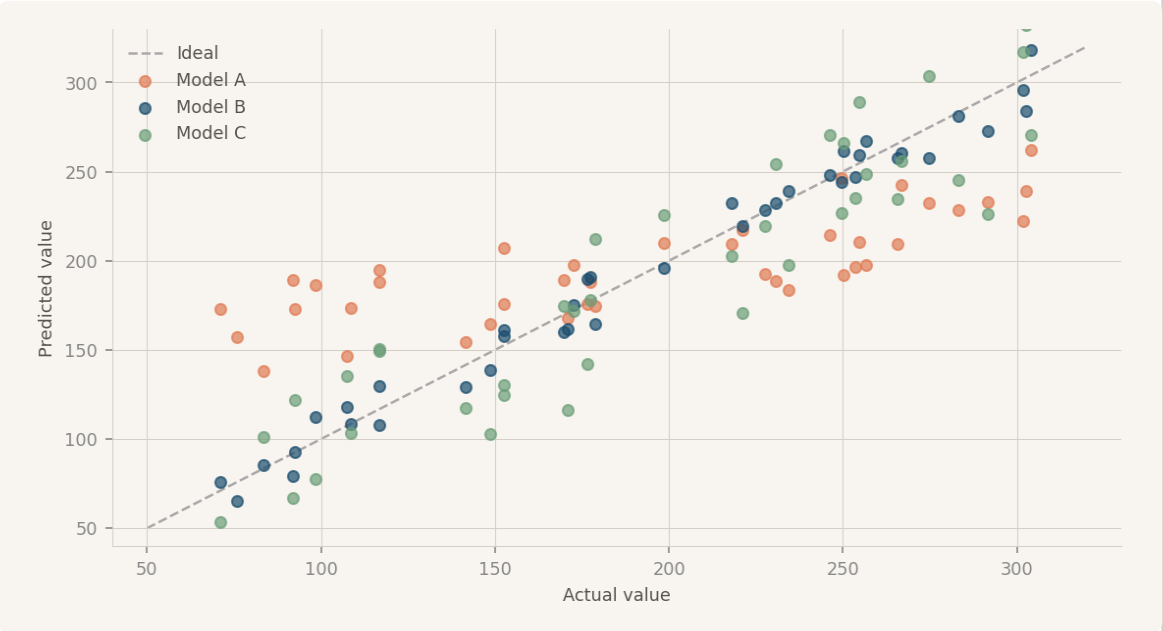

Figure 2.
Predicted versus actual disease progression values on the held-out test set. A perfect model would align along the dashed diagonal. Model B's predictions cluster most tightly around the diagonal. Model A shows a compressed spread, reflecting limited capacity. Model C shows greater scatter, particularly at the extremes of the target range.

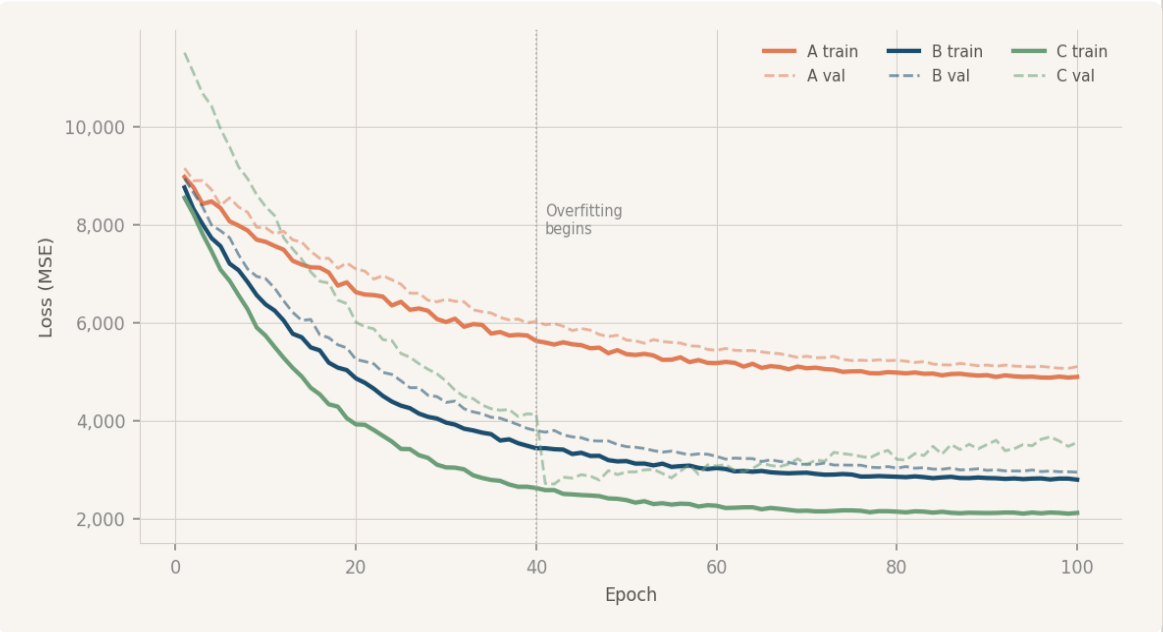

Figure 3.
Training and validation loss over 100 epochs. Model A converges quickly to a high plateau (underfitting). Model B shows a smooth, parallel decline. Model C's training loss continues to fall while its validation loss rises after epoch 40 — the classic overfitting signature.

Summary of Observed Performance

Model A	Model A demonstrated consistently poor performance. Both its training and test errors were high throughout training, confirming that the model lacked the capacity to represent the relationship between input features and the target variable. This is the expected behaviour for an underfitting model.
Model B	Model B achieved the best overall generalisation. It obtained the lowest test MSE among the three architectures, and the gap between training and test performance was small — suggesting genuine learning rather than memorisation of noise.
Model C	Model C performed strongly on the training set but its test performance was worse, and the divergence between training and validation loss increased over epochs — a clear sign of overfitting. Its excess capacity allowed it to learn dataset-specific noise that did not transfer to unseen samples.

SECTION 6

Discussion

Why Model A Underfit

Model A's single hidden layer with eight neurons gave it only a very limited number of parameters. This is a classic case of insufficient model capacity. The network's representational power was too constrained to approximate the mapping from the ten input features to the continuous disease progression target. It settled on overly broad predictions — essentially averaging out the complexity of the data rather than capturing it.

Why Model B Generalised Best

Model B's two-layer architecture appears to have struck the right balance for this task. The network was expressive enough to learn non-trivial relationships in the diabetes dataset, but not so over-parameterised that it was prone to overfitting. Its training and validation losses decreased together and stabilised at similar levels — strong evidence of genuine learning. This aligns with the bias-variance trade-off: a model with low bias and low variance achieves good generalisation.

Why Model C Overfit

Model C introduced far more parameters than the task required. With roughly 13,000 trainable parameters trained on 354 samples, the network effectively memorised individual training examples — including noise. The divergence between training and validation loss over epochs is the diagnostic signature of this behaviour.

It is important to note that Model C is not inherently a bad architecture. Given more data, stronger regularisation (dropout or L2 weight decay), or early stopping, a deeper network could potentially outperform Model B. Depth alone does not determine quality.

> **KEY TAKEAWAY**
>
> Increasing network depth does not automatically improve performance. For a given dataset and task, there is an effective range of model capacity; models below this range underfit, and models above it risk overfitting. The optimal architecture is not the deepest one available, but the one best matched to the complexity of the data.

SECTION 7

Ethical Considerations

Although this tutorial is primarily technical in scope, it touches on a domain — predicting medical outcomes — where the real-world implications of poor model design are significant and deserve careful reflection.

An overfitted model that performs well on training data but generalises poorly to new patients could produce dangerously misleading predictions if deployed in a clinical setting. In healthcare, the cost of a wrong prediction is not just statistical — it directly affects people's wellbeing. This reinforces the importance of rigorous evaluation on held-out data, ideally on multiple independent cohorts, before any model is considered for clinical use.

Beyond model performance, there is also the question of dataset bias. The Diabetes dataset was collected from a specific patient population at a particular point in time. A model trained solely on this data could perform well for patients similar to those in the training set while performing poorly for underrepresented groups. Responsible practice requires actively examining training data for such biases.

More generally, fairness, reliability, and transparency in evaluation are prerequisites for trustworthy deployment — particularly in high-stakes domains such as healthcare, finance, and criminal justice.

SECTION 8

Conclusion

This tutorial investigated how the depth of a neural network affects its performance, using three MLP architectures evaluated on the scikit-learn Diabetes regression dataset. The results provided a clear illustration of the underfitting–overfitting spectrum.

The shallow model (Model A) consistently underfit the data. The medium-depth model (Model B) achieved the best generalisation. The deep model (Model C), while achieving lower training error, showed a widening gap between training and validation performance — indicative of overfitting.

- Networks that are too small may underfit, failing to learn from the data at all.
- Networks of moderate depth and width can generalise well when properly matched to the task.
- Networks that are too deep or too wide risk overfitting, particularly when training data is limited.
- Bigger models are not always better; the goal is appropriate capacity, not maximum capacity.

These lessons apply broadly across machine learning applications. Good network design requires understanding the complexity of the problem, the size of the available data, and the interaction between model capacity and generalisation.

SECTION 9

References

[1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830. Neural network documentation available at: scikit-learn.org

[2] Scikit-learn Contributors (2024). *Diabetes Dataset*. Scikit-learn Documentation. Available at: scikit-learn.org — Toy Datasets.

[3] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. [Chapter 5 covers neural networks; Chapter 3 covers model selection and the bias-variance trade-off.]

[4] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), pp. 303–314.

SECTION 10

GitHub Repository

The full code for this tutorial — including data loading, preprocessing, model definitions, training loops, and visualisation scripts — is available in the repository linked below.

SOURCE CODE & REPOSITORY

https://github.com/banothvenkannauk-afk/mlp-depth-tutorial.git